```
async loadIndex(path) {
    this.vectorStore = await HNSWLib.load(path, this.embeddings);
    this.retriever = this.vectorStore.asRetriever({
      k: 6,
    });
    console.log('Vector store loaded successfully');
}
```

This code snippet demonstrates the simplicity of creating a retriever. We load the previously saved vector store and apply the `asRetriever method`, which we can use to perform semantic search operations.

In the subsequent augmentation and generation layers, these retrieved document chunks will be integrated with the original query to provide prompts to the language model. Let's explore the details.

AUGMENTATION AND GENERATION

The augmentation and generation phases represent the final, transformative steps in the RAG workflow. This process combines the retrieved contextual documents with the original query to generate a precise, context-aware response using an LLM.

To perform this, we simply craft a prompt that instructs the LLM and then use the retriever to ask the LLM to generate a response based on the prompt instruction.

Listing 7.9 Performing augmentation of the response

```
async performRAG(query) {
    const prompt = ChatPromptTemplate
.fromTemplate(`
        Answer the question based only on the context provided.

        Context: {context}

        Question: {question}`);

    const chain = RunnableSequence.from([
      {
        context: this.retriever.pipe(formatDocs),
        question: new RunnablePassthrough(),
      },
      prompt,
      this.llm,AQ:
      new StringOutputParser(),
    ]);

    const response = await chain.invoke(query);
    const sourceDocuments =
await this.retriever.invoke(query);
    return {
      answer: response,
      sourceDocuments,
    };
  }
```

Creates a chat prompt template that instructs the model to answer based only on provided context

Creates a processing pipeline that retrieves, formats, and generates a response

Generates the final response

Tracks the source documents used in the generation